

Gebze Technical University

CSE344 SYSTEM PROGRAMMING

-

MIDTERM REPORT

Veysel Cemaloğlu

200104004107

Problem Overview

My task was to create a program with a client-server architecture. The server acts as a central hub, processing requests sent by multiple clients. It can handle a predefined maximum number of concurrent connections, placing new clients in a waiting queue if the limit is reached. Additionally, the program is designed to manage diverse commands, large file transfers, and potential conflicts arising from simultaneous requests (race conditions).

Solution

Connection between Server – Client:

Upon startup, the server establishes a dedicated directory and log file to manage client interactions. To facilitate communication, FIFOs (named pipes) were chosen as the optimal method. The server continuously reads the FIFO using a while loop. When a client initiates a connection request, it's added to a queue. If the queue reaches its capacity, new clients are placed on hold until a slot becomes available. Semaphores were implemented to effectively manage this waiting process. To handle each client efficiently, the server utilizes a forking approach. This creates a child process for each incoming connection, enabling dedicated service for individual clients.

Communication between parent and child:

Maintaining communication between the server and child processes presented a unique challenge. While they share the same underlying queue data structure, the child process starting shouldn't affect the parent queue. To address this, I implemented a shared memory segment. This segment stores a custom structure containing both the queue and a counter. When a client successfully dequeues an element from the queue (client dequeue), the child process updates the counter within the shared memory. This allows the parent process to stay informed about queue modifications, even after forking child processes.

Signals:

The server is equipped to handle specific signals: SIGINT, SIGSTP, and SIGUSR1. When the server receives a SIGINT signal (often triggered by pressing Ctrl+C), it gracefully exits by terminating all child processes and sending termination signals (kill signals) to connected clients. Additionally, the server utilizes SIGUSR1 for client communication. If a client attempts to connect using the "tryConnect" keyword and encounters a full queue, the server sends a SIGUSR1 signal back to the client, informing them of the temporary unavailability.

Log file:

Log file is created with server pid when server is started. It holds the connection informations and the all request that are given by client side.

Reading Request from Client:

Client initialization involves creating two dedicated FIFOs (named pipes): one for writing (clientWriterFd) and another for reading (clientReaderFd). Both the server and client open these FIFOs with the appropriate flags to establish communication channels. The system then enters an infinite loop on both ends. The client continuously prompts the user for input. Meanwhile, the server remains vigilant, reading data from the client's write FIFO (clientWriterFd). Conversely, the client writes its input to the server's read FIFO.

Server Side:

```
ssize_t bytes_read = read(clientWriterFd, buf, sizeof(buf))
```

Client Side:

```
write(clientWriterFd, command, strlen(command))
```

Sending Response to Client:

To facilitate the exchange of information between client and server, I designed a custom Response structure. This structure efficiently stores the actual response data within a field called "content". Additionally, a flag named "isFinished" plays a crucial role in managing chunked data transmission. The server can leverage this flag to determine if the received data represents a complete response or only a portion of a larger message.

```
struct Response {  
    char content[4096];  
    int isFinished;  
};
```

Upon receiving a request from the client, the child process meticulously parses the provided string to extract the intended command. To ensure efficient handling, I created a dedicated function for each supported command. The child process then invokes the appropriate function, supplying the parsed command parameters. In cases where the received string doesn't contain a valid command, the child process returns an error message to the client. To accommodate potentially chunked responses, the client-side utilizes a custom structure to interpret the data. This structure includes a flag that signals the completion of the entire response. The client enters a loop, iteratively retrieving data chunks until the flag indicates a complete message.

readF

To manage access to files during read operations, the child process employs a semaphore mechanism. This ensures exclusive access, preventing conflicts when multiple clients might attempt to read the same file concurrently. Since the program is designed to handle large files, a chunked reading approach is implemented. The child process reads the file in manageable portions. Each chunk's "isFinished" flag is set to 0, signaling the client-side that more data might follow. This allows the client to seamlessly receive and reassemble the entire response.

If the requested line number exists within the file, the function efficiently locates it by meticulously counting newline characters. However, if the specified line number is not found, the entire file content is transmitted to the client in chunks, ensuring all data is delivered. Finally, to maintain system resource integrity, the function diligently closes the file descriptor and releases the acquired semaphores upon completion.

writeT

NOT IMPLEMENTED

list

The list function leverages inter-process communication (IPC) through pipes to retrieve directory listings. It accomplishes this by first creating a pipe and then forking a child process. This child process utilizes the `execl` system call to execute the `ls` command within the specified directory. To capture the output of the `ls` command, the child process redirects it to the newly created pipe.

Meanwhile, the parent process remains vigilant, reading data from the pipe. This retrieved data, representing the directory listing, is meticulously copied into the `response.content` variable within a custom response structure. Once the entire listing has been received, the `response.isFinished` flag is set to 1, indicating a complete response. The parent process then transmits the entire response structure, including the directory listing, to the client through the `clientReaderFd` file descriptor.

upload

Similar to the list function, the upload function utilizes semaphores to guarantee exclusive access to the target file during the upload process. This prevents data corruption that could occur if multiple clients attempted to upload to the same file simultaneously.

The function meticulously verifies the validity of both the source file (located in the client's current working directory) and the destination file on the server-side. It can handle scenarios where the client might not have the specified file or if a file with the same name already exists on the server.

To facilitate the upload process, the function enters a loop. Within this loop, it iteratively reads manageable chunks of data from the source file using the `fpr` file descriptor. These data chunks are then meticulously written to the target file using the `fpw` file descriptor. Throughout the process, the `total_uploaded_bytes` variable diligently tracks the cumulative number of bytes successfully uploaded.

Upon completion of the upload, the `response.content` field within the custom response structure is meticulously cleared. Subsequently, a success message is meticulously copied into the `response.content` field. Finally, the `isFinished` flag is set to 1, signifying a complete transfer. The entire response structure, encapsulating the success message and other relevant information, is then transmitted to the client through the `clientReaderFd` file descriptor.

Download

The download function facilitates the transfer of files from the server's directory to the client's machine. Similar to the upload and list functions, it employs semaphores to guarantee exclusive access to the requested file during the download process. This prevents data corruption that could occur if multiple clients attempted to download the same file concurrently.

Upon initiating the download, the function meticulously crafts and transmits a response structure to the client, signifying the commencement of the file transfer. It then enters a loop, where it iteratively reads manageable chunks of data from the source file on the server. These data chunks are meticulously written to the designated target file on the client's machine.

Once the download is successfully completed, the function diligently constructs and transmits another response structure to the client. This response structure encapsulates the total number of bytes transferred during the download process, providing the client with valuable feedback. Finally, the function ensures proper resource management by releasing the acquired semaphore and meticulously cleaning up any temporary resources utilized during the download.

archServer

NOT IMPLEMENTED!

Test Cases

```
veysel@veysel-Yoga-Slim-7-14ARE05:~/Desktop/mt$ ./neHosServer deneme1 2
>> Server Started PID 10623...
>> waiting for clients...
>> Client PID 10661 connected as "client01"
>> Client PID 10700 connected as "client02"
>> Connection request PID 10725... Que FULL
|
```

Client01 connects server and runs some commands:

```
veysel@veysel-Yoga-Slim-7-14ARE05:~/Desktop/mt$ ./neHosClient Connect 10623
>> Waiting for Que.. Connection established:
>> Enter comment : help
Available comments are :
    help, list, readF, upload, download, quit, killServer
>> Enter comment : help list
list
    Sends a request to display the list of files in Servers directory
>> Enter comment : |
```

Client02 connects server and runs some commands:

```
veysel@veysel-Yoga-Slim-7-14ARE05:~/Desktop/mt$ ./neHosClient Connect 10623
>> Waiting for Que.. Connection established:
>> Enter comment : help readF
readF <file> <line #>
    display the #th line of the <file>, returns with an error if <file> does
    not exists
>> Enter comment : list
log_file_10623.txt
>> Enter comment :
```

Client03 waits queue to connect server:

```
veysel@veysel-Yoga-Slim-7-14ARE05:~/Desktop/mt$ ./neHosClient Connect 10623
>> Waiting for Que..
```

Quiting first client:

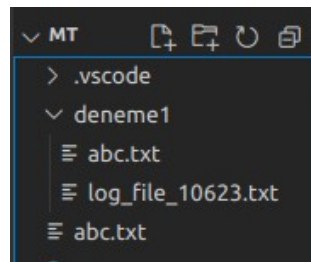
```
veysel@veysel-Yoga-Slim-7-14ARE05:~/Desktop/mt$ ./neHosClient Connect 10623
>> Waiting for Que.. Connection established:
>> Enter comment : help
Available comments are :
  help, list, readF, upload, download, quit, killServer
>> Enter comment : help list
list
      Sends a request to display the list of files in Servers directory
>> Enter comment : quit
veysel@veysel-Yoga-Slim-7-14ARE05:~/Desktop/mt$
```

Then client03 connects server and runs some commands:

```
veysel@veysel-Yoga-Slim-7-14ARE05:~/Desktop/mt$ ./neHosClient Connect 10623
>> Waiting for Que.. Connection established:
>> Enter comment : upload abc.txt
File transfer request received. Beginning file transfer:
53 bytes transferred

>> Enter comment : readF test.txt 1
Cannot open file
>> Enter comment :
```

You can see that readF not worked because there is no such file in directory (called deneme1) and abc.txt successfully uploaded:



Meanwhile at server side:

```
veysel@veysel-Yoga-Slim-7-14ARE05:~/Desktop/mt$ ./neHosServer deneme1 2
>> Server Started PID 10623...
>> waiting for clients...
>> Client PID 10661 connected as "client01"
>> Client PID 10700 connected as "client02"
>> Connection request PID 10725... Que FULL
>> client01 disconnected..
>> Client PID 10725 connected as "client03"
```


After quitting client01, trying to connect server back which is full due to client02 and client03 using tryConnect (this option leaves without waiting if the Que is full):

```
veysel@veysel-Yoga-Slim-7-14ARE05:~/Desktop/mt$ ./neHosClient Connect 10623
>> Waiting for Que.. Connection established:
>> Enter comment : help
Available comments are :
  help, list, readF, upload, download, quit, killServer
>> Enter comment : help list
list
    Sends a request to display the list of files in Servers directory
>> Enter comment : quit
veysel@veysel-Yoga-Slim-7-14ARE05:~/Desktop/mt$ ./neHosClient tryConnect 10623
veysel@veysel-Yoga-Slim-7-14ARE05:~/Desktop/mt$ |
```

Can not connect and leaves directly!

Client03 reads some lines from abc.txt:

```
veysel@veysel-Yoga-Slim-7-14ARE05:~/Desktop/mt$ ./neHosClient Connect 10623
>> Waiting for Que.. Connection established:
>> Enter comment : upload abc.txt
File transfer request received. Beginning file transfer:
53 bytes transferred

>> Enter comment : readF test.txt 1
Cannot open file
>> Enter comment : readF abc.txt 1
asdkjf

>> Enter comment : readF abc.txt 2
asdlgjabwelbawe

>> Enter comment :
```

```
deneme1 > ≡ abc.txt
1  asdkjf
2  asdlgjabwelbawe
3
4  |
5  qwo1p21u3414_!'^12
6  aw8eyr
7
```

```

veysel@veysel-Yoga-Slim-7-14ARE05:~/Desktop/mt$ ./neHosClient Connect 10623
>> Waiting for Que.. Connection established:
>> Enter comment : upload abc.txt
File transfer request received. Beginning file transfer:
53 bytes transferred

>> Enter comment : readF test.txt 1
Cannot open file
>> Enter comment : readF abc.txt 1
asdkjf

>> Enter comment : readF abc.txt 2
asdlgjabwelbawe

>> Enter comment : readF log_file_9052.txt 1
Cannot open file
>> Enter comment : readF log_file_10623.txt 1
>> Client PID 10661 connected as "client01"

>> Enter comment :

```

Client02 downloads log file from server:

```

veysel@veysel-Yoga-Slim-7-14ARE05:~/Desktop/mt$ ./neHosClient Connect 10623
>> Waiting for Que.. Connection established:
>> Enter comment : help readF
readF <file> <line #>
    display the #th line of the <file>, returns with an error if <file> does
    not exists
>> Enter comment : list
log_file_10623.txt

>> Enter comment : download log_file_10623.txt
File transfer request received. Beginning file transfer:
370 bytes transferred

>> Enter comment :

```

Client02 kills server:

```
veysel@veysel-Yoga-Slim-7-14ARE05:~/Desktop/mt$ ./neHosClient Connect 10623
>> Waiting for Que.. Connection established:
>> Enter comment : help readF
readF <file> <line #>
    display the #th line of the <file>, returns with an error if <file> does
    not exists
>> Enter comment : list
log_file_10623.txt

>> Enter comment : download log_file_10623.txt
File transfer request received. Beginning file transfer:
370 bytes transferred

>> Enter comment : help
Available comments are :
    help, list, readF, upload, download, quit, killServer
>> Enter comment : help killServer
killServer
    Sends a kill request to the Server
>> Enter comment : killServer
veysel@veysel-Yoga-Slim-7-14ARE05:~/Desktop/mt$ |
```

This kills client03 too:

```
>> Enter comment : readF log_file_9052.txt 1
Cannot open file
>> Enter comment : readF log_file_10623.txt 1
>> Client PID 10661 connected as "client01"

>> Enter comment : veysel@veysel-Yoga-Slim-7-14ARE05:~/Desktop/mt$
```

Server side:

```
veysel@veysel-Yoga-Slim-7-14ARE05:~/Desktop/mt$ ./neHosServer deneme1 2
>> Server Started PID 10623...
>> waiting for clients...
>> Client PID 10661 connected as "client01"
>> Client PID 10700 connected as "client02"
>> Connection request PID 10725... Que FULL
>> client01 disconnected..
>> Client PID 10725 connected as "client03"
>> kill signal from client02.. terminating...
>> bye...
>> client02 disconnected..
>> client03 disconnected..
```

Log File:

```
deneme1 > log_file_10623.txt
1  |> Client PID 10661 connected as "client01"
2  Comments:
3  help
4  help list
5  >> Client PID 10700 connected as "client02"
6  Comments:
7  help readF
8  list
9  quit
10 >> client01 disconnected..
11 >> Client PID 10725 connected as "client03"
12 Comments:
13 upload abc.txt
14 readF test.txt 1
15 readF abc.txt 1
16 readF abc.txt 2
17 readF log_file_9052.txt 1
18 readF log_file_10623.txt 1
19 download log_file_10623.txt
20 help
21 help killServer
22 killServer
23 >> client02 disconnected..
24 >> client03 disconnected..
25 End of the Server
```

Compiling and Running

Compile the code:

```
make
```

Run the Server:

```
./neHosServer <dirname> <maxclient>
```

Run a Client:

```
./neHosClient <Connect/tryConnect> <serverPID>
```